

LEZIONE 25/08/2026

1- NODE

Node.js è un runtime system open source multiplatforma orientato agli eventi per l'esecuzione di codice JavaScript, costruito sul motore JavaScript V8 di Google Chrome. Viene usato per realizzare applicazioni Web in JavaScript, permettendoci di utilizzare questo linguaggio, tipicamente utilizzato nella "client-side", anche per la scrittura di applicazioni "server-side".

Accenno Storico

Nato nel 1995 ad opera di Brendan Eich, JavaScript è stato per oltre un decennio confinato all'interno dei browser per rendere dinamiche le pagine web. La svolta è arrivata nel 2009, quando Ryan Dahl ha preso il potente motore V8 di Chrome e lo ha estratto dal browser, creando Node.js e permettendo a JS di girare direttamente sui server.

Analisi Concettuale della Definizione

Ecco la scomposizione logica e dettagliata delle parole chiave della definizione:

- **Runtime System (Ambiente di Esecuzione):** JavaScript nativamente sa fare ben poco da solo: ha bisogno di un "ambiente" che gli fornisca gli strumenti per interagire con il mondo esterno. Il browser è un runtime (offre strumenti come il DOM o l'oggetto `window`). Node.js è un altro runtime che gli fornisce strumenti diversi (come l'accesso al File System o alla rete del server o l'oggetto `global`).
- **Open Source:** Significa che il codice sorgente di Node.js è pubblico, gratuito e mantenuto da una vasta community globale. Chiunque può analizzarlo, contribuire o proporre migliorie.
- **Multiplatforma:** Il codice Node.js che scrivi sul tuo computer (sia esso macOS, Windows o Linux) eseguirà nello stesso identico modo su qualsiasi altro sistema operativo o server su cui è installato Node.js.
- **Orientato agli Eventi (Event-Driven):** Node.js funziona tramite un ciclo continuo chiamato *Event Loop*. Anziché bloccare l'esecuzione del programma mentre aspetta una risposta (es. la lettura di un file dal disco), registra un evento e continua a fare altro. Quando la risposta è pronta, scatena una funzione di callback.
- **Motore JavaScript V8:** È l'incredibile motore sviluppato da Google in C++ per Chrome. Prende il codice JavaScript (leggibile dagli umani) e lo compila direttamente in codice macchina (0 e 1, eseguibili dal processore) ad altissima velocità.
Nota: il motore **V8 non è semplicemente un compilatore**, ma un **sistema completo di esecuzione (engine)** che contiene al suo interno sia un interprete che un sistema di compilazione multilivello (ved. Approfondimento: V8 sotto).
- **Client-side vs Server-side:**
 - *Client-side:* Il codice gira sul dispositivo dell'utente (nel browser del PC o dello smartphone).
 - *Server-side:* Il codice gira sulla macchina remota (il server) che ospita il sito web, gestisce i database, autentica gli utenti ed elabora la logica aziendale

(Business Logic, ved. Approfondimento: Business Logic sotto) prima di inviare i dati al client.

Comando base: node <nome file js> - esegue il codice contenuto nel file js specificato all'interno del terminale.

2 - NPM e NODE

Quando installiamo node, viene installato anche un piccolo software di nome "npm" (Node Package Manager).

Come suggerisce la parola, è un "gestore di pacchetti".

Questo software ci libera dalla responsabilità di importare librerie e plugin vari manualmente (finora, ad esempio, abbiamo usato le CDN) e li gestisce per noi.

NPM genera un file package.json: un file chiave utilizzato in un progetto Node.js per descrivere le informazioni sul progetto stesso, le sue dipendenze, i comandi di script personalizzati e altre configurazioni. È un elemento fondamentale per la gestione dei pacchetti e la configurazione dei progetti [Node.js](#).

<u>COMANDO</u>	<u>DESCRIZIONE</u>
npm init	inizializzazione di NPM - Ci verranno fatte tutta una serie di domande che genereranno il file di configurazione base, package.json.
npm install <nome-pacchetto> (o npm i)	installazione dei pacchetti / librerie di terzi necessarie per il progetto
npm run start (nota per lo script "start" non è necessario scrivere "run", per tutti gli altri sì).	esegue il codice riportato dopo start nella sezione scripts del file package.json - es. "start": "node index.js " eseguirà il file index.js con node (nel terminale)
npm run watch	esegue il codice riportato dopo watch nella sezione scripts - es. "watch": "node -watch index.js " eseguirà il file index.js con node (nel terminale), <u>monitorando le modifiche in tempo reale (parola chiave watch dopo node e con i due trattini)</u>

Importante: una volta installato un pacchetto / libreria dobbiamo importarlo per poterlo utilizzare. Abbiamo a disposizione due sintassi:

CommonJS

```
const axios = require('axios');
```

ES6 Modules

```
import axios from 'axios'
```

Appendix- APPROFONDIMENTI

a- V8

Il percorso che il codice JavaScript compie all'interno di V8:

1. Parser: V8 legge il testo del codice JavaScript e lo trasforma in un albero concettuale chiamato *AST (Abstract Syntax Tree)*.
2. Interprete (chiamato *Ignition*): V8 prende l'AST e lo converte rapidamente in *bytecode* (un codice intermedio) per iniziare a eseguirlo immediatamente senza attese.
3. Compilatore JIT (chiamato *TurboFan*): Mentre l'interprete esegue il codice, V8 monitora quali parti vengono eseguite più spesso (le cosiddette "hot functions"). Il compilatore prende queste parti, le ottimizza e le compila direttamente in codice macchina (0 e 1 per la CPU).

JavaScript si definisce un linguaggio JIT (Just-In-Time compiled): non viene compilato *prima* dell'esecuzione (come C o Rust), ma viene interpretato e compilato *durante* l'esecuzione stessa.

Vantaggio chiave: la riduzione dei tempi morti all'avvio. L'interprete inizia a eseguire il codice **all'istante**, senza dover attendere che l'intero programma sia compilato prima di partire. Per una pagina web o una serverless function in Node.js, l'avvio immediato è fondamentale.

Svantaggi principali:

1. Consumo di memoria (RAM): Tenere in memoria sia l'interprete che il compilatore JIT insieme al codice macchina ottimizzato richiede più risorse.
2. Prestazioni iniziali: Nelle primissime fasi di esecuzione, il codice interpretato è più lento rispetto al codice pre-compilato di C++. Solo dopo che la funzione diventa "hot", il compilatore JIT la velocizza portandola a prestazioni quasi pari a C++.

b- Business Logic

Nel gergo dello sviluppo software, il termine **"logica aziendale"** (in inglese *Business Logic*, detta anche *Domain Logic*) indica **l'insieme di regole, calcoli e decisioni che definiscono come l'applicazione deve comportarsi per risolvere il problema per cui è stata creata**, indipendentemente dall'interfaccia grafica e dal database.

Analisi Concettuale: Cos'è la Business Logic?

Per capire il concetto, separa un'applicazione in tre livelli principali:

1. **Interfaccia Utente (Presentation Layer):** Ciò che l'utente vede e clicca (pulsanti, form HTML, stile CSS).
2. **Database (Data Layer):** Dove risiedono i dati grezzi (tabelle, utenti, prodotti).
3. **Logica Aziendale (Business Logic Layer):** Il "cervello" dell'applicazione. Riceve gli input, applica le regole e decide cosa fare.

Il termine "aziendale" o "business" è un'eredità storica dell'ingegneria del software, nata quando la maggior parte dei sistemi informatici veniva sviluppata per digitalizzare processi aziendali (contabilità, gestione magazzino). Oggi, nel mondo dello sviluppo web, **"Business Logic"** significa semplicemente **"Logica di Dominio"** o **"Regole del Gioco"**.